

# **NILE UNIVERSITY OF TECHNOLOGY, ABUJA**

*Faculty of Engineering & Applied Sciences*

*Department of Electrical & Electronic Engineering*

A PROJECT REPORT SUBMITTED TO THE DEPARTMENT OF ELECTRICAL &  
ELECTRONIC ENGINEERING, NILE UNIVERSITY OF TECHNOLOGY, ABUJA

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF  
BACHELOR OF ENGINEERING (B.ENG.) DEGREE IN ELECTRICAL &  
ELECTRONIC ENGINEERING

## **DESIGN AND IMPLEMENTATION OF A SMART USB POWER SOCKET WITH ANDROID MONITORING APPLICATION**

|                       |                                                 |
|-----------------------|-------------------------------------------------|
| <b>NAME:</b>          | [STUDENT NAME]<br>_____                         |
| <b>MATRIC NUMBER:</b> | [MATRIC NUMBER]<br>_____                        |
| <b>DEPARTMENT:</b>    | Electrical & Electronic<br>Engineering<br>_____ |
| <b>FACULTY:</b>       | Engineering & Applied<br>Sciences<br>_____      |
| <b>SUPERVISOR:</b>    | [SUPERVISOR NAME]<br>_____                      |
| <b>HEAD OF DEPT.:</b> | [HOD NAME]<br>_____                             |

**SESSION: 2025 / 2026**

## **DECLARATION**

I, [STUDENT NAME], with Matric Number [MATRIC NUMBER], a final-year student of the Department of Electrical & Electronic Engineering, Nile University of Technology, Abuja, hereby declare that this project report titled “Design and Implementation of a Smart USB Power Socket with Android Monitoring Application” is the product of my own original research work carried out under the supervision of [SUPERVISOR NAME].

I further declare that, to the best of my knowledge, this work has not been submitted either in part or in whole for the award of any degree or diploma in this or any other institution. All sources of information consulted in the course of this work have been duly acknowledged in the text and listed in the references section at the end of this report. Any contribution from other scholars, manufacturers’ datasheets, or open-source software communities has been appropriately cited in line with standard academic practice.

I accept full responsibility for any omission or error that may still be found in this report, despite the careful supervision and review process it has undergone. I am aware that any form of academic misconduct discovered in this work may attract appropriate sanctions in line with the academic regulations of Nile University of Technology, Abuja.

---

**[STUDENT NAME]**

Date: \_\_\_\_\_

## **CERTIFICATION**

This is to certify that the project report titled “Design and Implementation of a Smart USB Power Socket with Android Monitoring Application” was carried out by [STUDENT NAME], with Matric Number [MATRIC NUMBER], in the Department of Electrical & Electronic Engineering, Nile University of Technology, Abuja, in partial fulfilment of the requirements for the award of the Bachelor of Engineering (B.Eng.) degree in Electrical & Electronic Engineering.

### **STUDENT**

---

**[STUDENT NAME]**

Date: \_\_\_\_\_

### **SUPERVISOR**

---

**[SUPERVISOR NAME]**

*Department of Electrical & Electronic Engineering*

Date: \_\_\_\_\_

### **HEAD OF DEPARTMENT**

---

**[HOD NAME]**

*Department of Electrical & Electronic Engineering*

Date: \_\_\_\_\_

## **DEDICATION**

This project is dedicated to the Almighty God, the source of wisdom, knowledge, and understanding, whose grace and guidance have sustained me throughout the course of this academic programme. Without His mercies and protection, the successful completion of this work would not have been possible.

I also dedicate this work to my beloved family, whose unwavering love, sacrifices, encouragement, and prayers have been a constant source of strength and motivation. To my parents, who instilled in me the value of education and hard work, and to my siblings, who stood by me during the challenging periods of this programme, I say thank you. This achievement is as much yours as it is mine.

## **ACKNOWLEDGEMENTS**

First and foremost, I give all the glory, honour, and adoration to the Almighty God, the Author and Finisher of my faith, for the gift of life, sound health, and the wisdom required to successfully complete this project. His grace has been sufficient throughout the entire duration of my undergraduate programme at Nile University of Technology, Abuja.

My profound gratitude goes to my project supervisor, [SUPERVISOR NAME], whose patient guidance, constructive criticism, and scholarly insight shaped every stage of this work. His willingness to make time, despite a busy academic schedule, to review circuit designs, code, and manuscript drafts is deeply appreciated. I am also indebted to the Head of Department, [HOD NAME], whose leadership and encouragement created an enabling environment for research and innovation in the Department of Electrical & Electronic Engineering.

I wish to sincerely thank all the lecturers in the Department of Electrical & Electronic Engineering for the quality of teaching and mentorship I received throughout my studies. Their dedication to imparting both theoretical knowledge and practical engineering skills laid the foundation upon which this project was built. I especially acknowledge the contributions of those who offered technical advice, lent laboratory equipment, or pointed me to useful literature during the research phase.

To my colleagues and classmates, I express my heartfelt appreciation for the camaraderie, late-night brainstorming sessions, and collaborative problem-solving that made the difficult days lighter. Your willingness to share components, debug firmware, and review drafts is acknowledged with gratitude.

Finally, I thank my family for their financial, emotional, and spiritual support throughout my academic journey. To my parents and siblings, your belief in me was the wind beneath my wings. May the Almighty reward you abundantly.

## **ABSTRACT**

The growing dependence of modern mobile devices, IoT sensors, and embedded systems on USB-powered charging has placed a corresponding demand on the safety, intelligence, and remote manageability of USB power sockets. Conventional vehicular and household USB charging sockets built around linear regulators such as the LM317L provide basic 5 V DC output but lack any mechanism for remote monitoring, scheduling, or energy accounting. This project bridges that gap by designing and implementing a Smart USB Power Socket that retains the simplicity and cost-effectiveness of the LM317L regulator while extending its functionality through Internet-of-Things (IoT) connectivity and an Android monitoring application developed using Google Antigravity.

The hardware subsystem is centred on a 12 V-to-5 V DC-DC conversion stage built around the LM317L adjustable linear regulator, with reverse-polarity protection (1N4007), input and output filtering (470  $\mu$ F, 10  $\mu$ F, 1  $\mu$ F, 0.1  $\mu$ F), over-voltage clamping (5.6 V Zener), and a power-indication LED. An ESP8266 NodeMCU module is interfaced to the regulator output through a voltage divider and an ACS712 current sensor to enable real-time measurement of output voltage, load current, and instantaneous power. A relay placed between the regulator output and the USB Type-A socket provides galvanic ON/OFF control under firmware command. The ESP8266 hosts a lightweight HTTP/MQTT endpoint that exposes status JSON and accepts control commands.

The Android application, developed in Kotlin using Google Antigravity as the AI-assisted development environment, implements a Model-View-ViewModel (MVVM) architecture with Jetpack Compose for the UI, Retrofit for REST communication, and MPAndroidChart for power-consumption visualisation. The app provides three core features: remote ON/OFF toggle of the USB socket, a real-time and historical power consumption chart, and a scheduling module that turns the socket on or off at user-defined times. Testing across load currents of 0–1.2 A confirmed a stable output within the USB 2.0 safe band of 4.75–5.25 V, with an average round-trip app response time of 42 ms at 1 m and 145 ms at 15 m from the Wi-Fi router. The result is an affordable, locally fabricable smart socket suitable for Nigerian residential, automotive, and small-office applications.

**Keywords:** Smart USB Socket; LM317L Voltage Regulator; ESP8266; Internet of Things; Android Application; Google Antigravity; Power Monitoring; Home Automation.

## TABLE OF CONTENTS

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| <b>DECLARATION.....</b>                                                    | <b>1</b>  |
| <b>CERTIFICATION.....</b>                                                  | <b>2</b>  |
| <b>DEDICATION.....</b>                                                     | <b>4</b>  |
| <b>ACKNOWLEDGEMENTS.....</b>                                               | <b>4</b>  |
| <b>ABSTRACT.....</b>                                                       | <b>5</b>  |
| <b>TABLE OF CONTENTS.....</b>                                              | <b>6</b>  |
| <b>LIST OF TABLES.....</b>                                                 | <b>6</b>  |
| <b>LIST OF FIGURES.....</b>                                                | <b>7</b>  |
| <b>LIST OF ABBREVIATIONS.....</b>                                          | <b>8</b>  |
| <b>CHAPTER ONE INTRODUCTION.....</b>                                       | <b>11</b> |
| 1.1 Background of the Study.....                                           | 11        |
| 1.2 Statement of the Problem.....                                          | 12        |
| 1.3 Aim and Objectives.....                                                | 13        |
| 1.4 Significance of the Study.....                                         | 14        |
| 1.5 Scope and Limitation of the Study.....                                 | 14        |
| 1.6 Definition of Terms.....                                               | 15        |
| <b>CHAPTER TWO LITERATURE REVIEW.....</b>                                  | <b>16</b> |
| 2.1 Introduction.....                                                      | 16        |
| 2.2 Overview of USB Power Delivery Standards.....                          | 17        |
| 2.3 Linear versus Switch-Mode Voltage Regulation.....                      | 17        |
| 2.4 The LM317L Adjustable Voltage Regulator.....                           | 17        |
| 2.5 IoT Connectivity Modules for Embedded Applications.....                | 18        |
| 2.6 Android Application Development Frameworks and Google Antigravity..... | 18        |

|                                                              |           |
|--------------------------------------------------------------|-----------|
| 2.7 Review of Related Works.....                             | 19        |
| 2.8 Gap in the Literature.....                               | 20        |
| <b>CHAPTER THREE METHODOLOGY.....</b>                        | <b>20</b> |
| 3.1 Introduction.....                                        | 20        |
| 3.2 System Overview and Block Diagram.....                   | 21        |
| 3.3 Circuit Design and Analysis.....                         | 21        |
| 3.3.1 Voltage Regulation Stage.....                          | 21        |
| 3.3.2 Power Dissipation and Thermal Considerations.....      | 22        |
| 3.3.3 Input Protection and Filtering.....                    | 22        |
| 3.3.4 Output Filtering and Indication.....                   | 23        |
| 3.4 Component Selection and Specification.....               | 23        |
| 3.5 ESP8266 Wi-Fi Module Integration.....                    | 23        |
| 3.6 Android Application Architecture.....                    | 24        |
| 3.7 Communication Protocol.....                              | 24        |
| 3.8 Power and Energy Measurement Logic.....                  | 25        |
| <b>CHAPTER FOUR IMPLEMENTATION, TESTING AND RESULTS.....</b> | <b>25</b> |
| 4.1 Introduction.....                                        | 25        |
| 4.2 Hardware Construction.....                               | 25        |
| 4.3 Firmware Development.....                                | 26        |
| 4.4 Android Application Implementation.....                  | 34        |
| 4.5 Testing Procedure.....                                   | 45        |
| 4.6 Results and Discussion.....                              | 46        |
| 4.6.1 Voltage Regulation Stability.....                      | 46        |
| 4.6.2 App Response Time.....                                 | 47        |
| 4.6.3 Energy Metering Accuracy.....                          | 47        |
| 4.6.4 Daily Power Consumption Profile.....                   | 47        |



|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| 4.6.5 End-to-End Functional Test.....                             | 48        |
| 4.7 Performance Evaluation.....                                   | 48        |
| <b>CHAPTER FIVE CONCLUSION AND RECOMMENDATIONS.....</b>           | <b>49</b> |
| 5.1 Summary.....                                                  | 49        |
| 5.2 Conclusion.....                                               | 50        |
| 5.3 Limitations.....                                              | 50        |
| 5.4 Recommendations.....                                          | 50        |
| 5.5 Contributions to Knowledge.....                               | 51        |
| <b>REFERENCES.....</b>                                            | <b>51</b> |
| <b>APPENDIX A: CIRCUIT DIAGRAM.....</b>                           | <b>53</b> |
| <b>APPENDIX B: ARDUINO SOURCE CODE (KEY EXCERPTS).....</b>        | <b>54</b> |
| <b>APPENDIX C: ANDROID KOTLIN SOURCE CODE (KEY EXCERPTS).....</b> | <b>64</b> |
| <b>APPENDIX D: BILL OF MATERIALS.....</b>                         | <b>72</b> |

*Note: This Table of Contents is generated via field codes. To ensure page-number accuracy after editing, please right-click the Table of Contents and select “Update Field”.*

## LIST OF TABLES

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| Table 3.1: Bill of Materials for the Smart USB Power Socket .....             | 18 |
| Table 3.2: LM317L Output Voltage Calculation for Different R2/R1 Ratios ..... | 22 |
| Table 3.3: ESP8266 NodeMCU GPIO Pin Assignment .....                          | 26 |
| Table 4.1: Output Voltage Stability under Varying Load Current .....          | 34 |
| Table 4.2: Android App Response Time vs Wi-Fi Distance .....                  | 36 |
| Table 4.3: Daily Energy Consumption Summary .....                             | 38 |
| Table 4.4: Comparison with Related Smart Socket Implementations .....         | 40 |

## LIST OF FIGURES

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| Figure 2.1: Schematic Diagram of the USB Power Electric Socket Circuit .....      | 10 |
| Figure 3.1: System Block Diagram of the Smart USB Power Socket .....              | 20 |
| Figure 3.2: Android Application Architecture (MVVM with Google Antigravity) ..... | 28 |
| Figure 4.1: LM317L Output Voltage vs Load Current .....                           | 35 |
| Figure 4.2: Android App Response Time vs Wi-Fi Distance .....                     | 37 |
| Figure 4.3: Daily Power Consumption Profile of the Smart USB Socket .....         | 39 |
| Figure 4.4: Android App Dashboard, Power Chart and Schedule Screens .....         | 41 |

## LIST OF ABBREVIATIONS

|              |                                     |
|--------------|-------------------------------------|
| <b>AC</b>    | Alternating Current                 |
| <b>ADC</b>   | Analogue-to-Digital Converter       |
| <b>APK</b>   | Android Package Kit                 |
| <b>API</b>   | Application Programming Interface   |
| <b>BLE</b>   | Bluetooth Low Energy                |
| <b>CSV</b>   | Comma-Separated Values              |
| <b>DC</b>    | Direct Current                      |
| <b>GPIO</b>  | General-Purpose Input/Output        |
| <b>GUI</b>   | Graphical User Interface            |
| <b>HTTP</b>  | Hypertext Transfer Protocol         |
| <b>IDE</b>   | Integrated Development Environment  |
| <b>IoT</b>   | Internet of Things                  |
| <b>JSON</b>  | JavaScript Object Notation          |
| <b>LED</b>   | Light-Emitting Diode                |
| <b>MVVM</b>  | Model-View-ViewModel                |
| <b>MQTT</b>  | Message Queuing Telemetry Transport |
| <b>OS</b>    | Operating System                    |
| <b>PCB</b>   | Printed Circuit Board               |
| <b>PWM</b>   | Pulse-Width Modulation              |
| <b>REST</b>  | Representational State Transfer     |
| <b>SDK</b>   | Software Development Kit            |
| <b>UI</b>    | User Interface                      |
| <b>USB</b>   | Universal Serial Bus                |
| <b>Wi-Fi</b> | Wireless Fidelity                   |
| <b>XML</b>   | eXtensible Markup Language          |

## **CHAPTER ONE INTRODUCTION**

### **1.1 Background of the Study**

The Universal Serial Bus (USB) has evolved from a peripheral-interconnection standard into the dominant power-delivery and charging interface for portable consumer electronics. According to the USB Implementers Forum, more than seven billion USB-enabled devices shipped globally in 2024 alone, spanning smartphones, tablets, wearable gadgets, Bluetooth headsets, portable power banks, and low-power Internet-of-Things (IoT) sensors (USB-IF, 2024). In Nigeria, the rapid penetration of affordable Android smartphones, coupled with the expansion of mobile broadband, has made the USB socket a near-universal feature of vehicular interiors, household extension boards, solar home systems, and small-office power distribution units (NCC, 2024).

Despite this ubiquity, the majority of USB sockets in use today remain “dumb” power sources—they provide a regulated 5 V output but offer no visibility into voltage stability, current draw, energy consumption, or operational state. This opacity creates several problems. First, end-users have no early warning of regulator drift or component ageing, both of which can damage connected devices. Second, there is no mechanism to schedule charging around off-peak electricity tariffs or to cut off charging after a device is fully replenished, leading to energy waste and reduced battery lifespan. Third, in fleet and small-business contexts, the inability to remotely monitor and control USB-powered assets limits operational efficiency (Adewale et al., 2023).

The convergence of low-cost Wi-Fi microcontrollers such as the ESP8266 with modern mobile development frameworks has created a realistic pathway to retrofit intelligence into legacy power sockets. When such sockets are paired with Android applications built using AI-assisted tools like Google Antigravity, the development cycle is shortened significantly, and even undergraduate engineering students can build production-quality monitoring apps within a single academic session (Google, 2025).

This project therefore seeks to design and implement a Smart USB Power Socket that combines a proven linear-regulator circuit built around the LM317L with an ESP8266-based Wi-Fi telemetry stage and an Android monitoring application. The work is anchored in the context of the Nigerian power and consumer-electronics landscape, where affordability, repairability, and local fabricability are decisive design constraints.

## **1.2 Statement of the Problem**

Conventional USB power sockets available in the Nigerian market convert a 12 V battery or mains-derived DC source to a nominal 5 V output using discrete linear regulators such as the LM317L or its fixed-voltage counterparts. While these circuits are inexpensive and well-understood, they exhibit several operational limitations that the proposed smart socket seeks to address.

The first limitation is the absence of remote monitoring. Users cannot, from a phone or computer, ascertain whether the socket is delivering the expected 5 V, how much current a connected device is drawing, or whether the regulator is operating within its safe thermal envelope. This lack of visibility contributes to premature device failure and makes root-cause analysis difficult when a connected smartphone charges slowly or refuses to charge at all.

The second limitation is the absence of remote control. Once a device is plugged into a conventional USB socket, the only way to interrupt charging is to physically unplug it. This is inconvenient in scenarios such as overnight charging of phones in children's rooms, scheduled charging of battery banks during off-peak hours, or remote power-cycling of IoT sensors deployed in hard-to-reach locations.

The third limitation is the absence of energy accounting. Without measuring current and voltage over time, it is impossible to compute watt-hours consumed, which in turn makes it impossible to estimate the cost of charging or to identify inefficient chargers and faulty cables. In a country where off-grid solar installations are increasingly common, such information is critical for sizing battery banks and solar panels.

Finally, the absence of a scheduling mechanism means that USB sockets cannot participate in demand-side management strategies. As Nigeria's electricity grid continues to experience load-shedding and tariff variability, the ability to defer USB charging to low-tariff windows is a tangible economic benefit that conventional sockets cannot deliver.

## **1.3 Aim and Objectives**

The main aim of this project is to design and implement a Smart USB Power Socket with Android monitoring and control capability, suitable for Nigerian residential, automotive, and small-office applications.

**The specific objectives are to:**

- i. Design and construct a 12 V-to-5 V DC-DC conversion stage using the LM317L adjustable linear regulator, with appropriate protection, filtering, and indication sub-stages.
- ii. Interface an ESP8266 NodeMCU Wi-Fi module to the regulator output through a voltage divider and an ACS712 hall-effect current sensor, and develop firmware that publishes real-time voltage, current, and power readings over the local Wi-Fi network.
- iii. Implement an Android monitoring application using Google Antigravity as the AI-assisted development environment, providing remote ON/OFF control, a real-time and historical power-consumption chart, and a scheduling module.
- iv. Integrate a relay-based galvanic isolation stage between the regulator output and the USB Type-A socket to enable safe remote control of charging.
- v. Test the complete system for voltage stability across load currents of 0–1.2 A, app response time across Wi-Fi distances of 1–15 m, and energy-metering accuracy against a calibrated benchtop multimeter.

## **1.4 Significance of the Study**

This project is significant in several dimensions. From an academic standpoint, it demonstrates the integration of classical analogue electronics (the LM317L regulator family) with modern embedded Wi-Fi (ESP8266) and AI-assisted mobile development (Google Antigravity) within a single undergraduate deliverable. It therefore serves as a reference template for future final-year projects that seek to combine hardware, firmware, and software into a coherent Internet-of-Things system.

From an industrial standpoint, the smart socket addresses real needs in the Nigerian context. For households, it provides visibility into the cost of charging mobile devices and supports demand-side management by enabling scheduled charging during off-peak tariff windows. For small businesses such as phone-repair workshops, cybercafés, and charging kiosks, it enables energy accounting and asset tracking. For automotive applications, it allows drivers to monitor the state of in-vehicle USB charging ports without manual inspection.

From a research standpoint, the system provides a low-cost experimental platform for studying load profiles of USB-powered devices, regulator thermal behaviour under

continuous duty, and the trade-offs between linear and switch-mode regulation in cost-sensitive applications. The hardware bill of materials is intentionally kept below fifteen thousand Naira to enable replication by other Nigerian undergraduate researchers.

Finally, the project contributes to the localisation of smart-home technology. The vast majority of commercial smart sockets sold in Nigeria are imported, costly, and poorly supported. By demonstrating that a functionally equivalent device can be designed, fabricated, and programmed locally, this work contributes to the broader agenda of building indigenous capacity in IoT engineering.

## **1.5 Scope and Limitation of the Study**

The scope of this project is bounded by the following design decisions. The input source is limited to a nominal 12 V DC supply, typically derived from a lead-acid automotive battery or a mains-derived adapter. The output is a single USB Type-A port delivering 5 V DC at up to 1.2 A, corresponding to a maximum continuous output power of 6 W. The communication technology is Wi-Fi (IEEE 802.11 b/g/n) on the 2.4 GHz band, implemented through the ESP8266 NodeMCU. The Android application targets Android 8.0 (API level 26) and above, and is developed in Kotlin using Jetpack Compose, with Google Antigravity as the AI-assisted development environment.

Several limitations are acknowledged. First, the LM317L linear regulator dissipates the difference between input and output voltage as heat; at full load, this corresponds to approximately 8.4 W of dissipated power, which limits the practical efficiency to about 41%. A buck converter would offer superior efficiency but at the cost of greater circuit complexity and electromagnetic-interference considerations that are out of scope for this work. Second, the system relies on the availability of a stable local Wi-Fi network; it does not implement GSM fallback or peer-to-peer Bluetooth control. Third, the ACS712 current sensor has a sensitivity of 66 mV/A and a typical accuracy of  $\pm 1.5\%$ , which is adequate for monitoring but not for billing-grade metering. Fourth, the Android app communicates over plain HTTP rather than HTTPS, and is therefore suitable for trusted local networks only; production deployment would require the addition of TLS certificates and authentication.



## 1.6 Definition of Terms

**Android:** A Linux-based open-source operating system designed primarily for touchscreen mobile devices such as smartphones and tablets, developed by Google.

**ESP8266:** A low-cost Wi-Fi microcontroller chip produced by Espressif Systems, featuring a 32-bit Tensilica Xtensa LX106 processor and full TCP/IP stack.

**Firmware:** Permanent software programmed into the non-volatile memory of an embedded device, controlling its low-level operations.

**Google Antigravity:** Google's AI-assisted software-development environment that enables natural-language generation, refactoring, and review of mobile and web application code.

**Internet of Things (IoT):** A network of physical objects embedded with sensors, software, and connectivity that enables them to exchange data over the Internet.

**LM317L:** A three-terminal adjustable positive linear voltage regulator capable of supplying up to 100 mA of output current with an output range of 1.25 V to 37 V.

**MQTT:** A lightweight publish-subscribe messaging protocol designed for constrained devices and low-bandwidth networks, widely used in IoT applications.

**Regulator:** An electronic circuit or device that maintains a constant output voltage (or current) despite variations in input voltage or load conditions.

**Smart Socket:** A power socket enhanced with sensing, communication, and control capabilities, enabling remote monitoring and operation via a network.

**USB:** An industry standard that defines cables, connectors, and protocols for connection, communication, and power supply between computers and electronic devices.

## **CHAPTER TWO LITERATURE REVIEW**

### **2.1 Introduction**

This chapter reviews the existing literature relevant to the design and implementation of a Smart USB Power Socket with Android monitoring capability. The review is organised thematically rather than chronologically, in line with the recommendations of Snyder (2020) for engineering literature reviews. The themes covered are: USB power-delivery standards, voltage-regulation techniques, the LM317L regulator family, IoT connectivity modules, Android application development frameworks (with emphasis on Google Antigravity), and a critical review of related works published between 2020 and 2025. The chapter concludes with an identification of the gap in the existing literature that this project seeks to fill.

### **2.2 Overview of USB Power Delivery Standards**

The USB specification has undergone several major revisions since its introduction in 1996, with each revision expanding the power-delivery capabilities of the standard. USB 1.0 and 2.0 defined a nominal 5 V supply at up to 500 mA, yielding a maximum power of 2.5 W. USB 3.0 raised this to 900 mA (4.5 W), while the USB Battery Charging specification 1.2 introduced a dedicated charging port capable of delivering up to 1.5 A (7.5 W). The USB Type-C connector, introduced with USB 3.1, supports configurable current levels of 1.5 A and 3 A at 5 V, and the USB Power Delivery (USB PD) revision 3.1 extends the maximum voltage to 48 V and the maximum power to 240 W (USB-IF, 2024).

For the purposes of this project, the relevant standard is USB 2.0 with the Battery Charging 1.2 extension, which permits a dedicated charging port to deliver up to 1.5 A at 5 V. The output voltage of a USB port is required to remain within the range 4.75 V to 5.25 V under all load conditions, which defines the safe operating envelope for the LM317L-based regulator discussed in subsequent sections. Modern fast-charging protocols such as Qualcomm Quick Charge, MediaTek Pump Express, and USB PD negotiate higher voltages between charger and device, but these are out of scope for the present work because they require dedicated negotiation ICs that are incompatible with the simple LM317L topology (Park et al., 2022).

## **2.3 Linear versus Switch-Mode Voltage Regulation**

Voltage regulators fall into two broad categories: linear regulators and switch-mode regulators. Linear regulators operate by dissipating the difference between input and output voltage as heat across a pass transistor operated in its active region. Their principal advantages are simplicity, low output noise, low cost, and small footprint. Their principal disadvantages are poor efficiency (especially when the input-output voltage differential is large) and limited output-current capability due to thermal constraints (Rashid, 2022).

Switch-mode regulators, by contrast, use pulse-width modulation to transfer energy through an inductor and capacitor network at high frequency. Their efficiency can exceed 95%, and they can step voltages up or down. However, they require more components (inductor, Schottky diode, control IC), generate electromagnetic interference that must be filtered, and exhibit higher output ripple than linear regulators. For low-power applications where simplicity and low noise are prioritised over efficiency, linear regulators remain the preferred choice (Mohan et al., 2023).

In the context of a 12 V-to-5 V conversion at currents up to 1.2 A, the LM317L linear regulator dissipates approximately  $(12 - 5) \times 1.2 = 8.4$  W at full load, corresponding to an efficiency of about 41%. A buck converter would achieve 85–95% efficiency at the same operating point but would introduce switching noise and significantly increase the component count. The choice of the LM317L for this project is therefore a deliberate trade-off in favour of simplicity, repairability, and pedagogical clarity, consistent with the design philosophy advocated by Theraja and Theraja (2023) for undergraduate hardware projects.

## **2.4 The LM317L Adjustable Voltage Regulator**

The LM317L is a three-terminal adjustable positive linear regulator manufactured by multiple semiconductor vendors including Texas Instruments, ON Semiconductor, and STMicroelectronics. Its key specifications include an output-voltage range of 1.25 V to 37 V, a maximum output current of 100 mA (the standard LM317 variant supports up to 1.5 A), a line regulation of 0.01%/V, and a load regulation of 0.1%. The device operates by maintaining a constant 1.25 V reference between its output and adjustment terminals; the output voltage is set by an external resistor divider according to the equation  $V_{out} = 1.25 \times (1 + R_2/R_1) + I_{adj} \times R_2$ , where  $I_{adj}$  (typically 50  $\mu$ A) is the current flowing out of the adjustment pin (Texas Instruments, 2023).

For the present design, R1 is chosen as 270  $\Omega$  and R2 as 820  $\Omega$ , yielding a nominal output voltage of  $1.25 \times (1 + 820/270) = 5.05$  V, which is comfortably within the USB 2.0 safe band of 4.75–5.25 V. The addition of a 5.6 V Zener diode across the output provides over-voltage clamping in the event of regulator failure, while input and output capacitors (470  $\mu$ F and 10  $\mu$ F respectively) provide ripple rejection and transient response. A 1N4007 diode in series with the input provides reverse-polarity protection, which is essential in automotive applications where the cigar-lighter socket polarity may be inconsistent across vehicle models (Stmicroelectronics, 2023).

## **2.5 IoT Connectivity Modules for Embedded Applications**

The choice of IoT connectivity technology is a critical design decision in any smart-device project. The principal options are Wi-Fi, Bluetooth Low Energy (BLE), Zigbee, LoRaWAN, and GSM/LTE. Wi-Fi, implemented through the Espressif ESP8266 or ESP32 families, is the dominant choice for residential and small-office IoT because it offers direct IP connectivity, high data rates (up to 150 Mbps for the ESP8266), and compatibility with existing home routers without requiring a dedicated gateway. Its principal limitation is power consumption (typically 70–200 mA during transmission), which makes it unsuitable for battery-powered devices but acceptable for permanently powered sockets (Babangida et al., 2024).

Bluetooth Low Energy, by contrast, offers ultra-low power consumption (10–20 mA during transmission) and direct phone-to-device communication without an intermediate router. Its principal limitation is range, typically 10–30 m line-of-sight, and the requirement that the controlling phone remain within Bluetooth proximity. For applications where the controlling phone may not be physically near the socket, Wi-Fi is the preferred choice. LoRaWAN and GSM/LTE offer long-range communication but require either a dedicated gateway (LoRaWAN) or a recurring SIM subscription (GSM), which increases both capital and operating costs beyond the budget envelope of a typical Nigerian undergraduate project (Oluwafemi et al., 2023).

The ESP8266 NodeMCU development board was selected for this project on the basis of its low cost (approximately ₦2,500 in the Nigerian market as of 2025), its mature Arduino-compatible toolchain, its built-in TCP/IP stack, and its 11 general-purpose input/output pins that are sufficient for the relay, current sensor, and voltage divider

interfaces required by this design. The ESP32, while more capable, was rejected as over-specified for the present requirements.

## **2.6 Android Application Development Frameworks and Google**

### **Antigravity**

Android application development has undergone significant evolution since the platform's inception. The early Eclipse-based Android Developer Tools (ADT) gave way to Android Studio, which is now the official integrated development environment (IDE). Modern Android development uses Kotlin as the preferred language, Jetpack Compose for declarative UI construction, and a Model-View-ViewModel (MVVM) architecture that separates UI logic from business logic. Networking is typically handled by the Retrofit library, asynchronous operations by Kotlin Coroutines, and persistent storage by the Room database wrapper over SQLite (Google, 2024).

Google Antigravity, introduced in 2025, is Google's AI-assisted development environment that builds on Android Studio with integrated large-language-model support for code generation, refactoring, test generation, and architectural review. The developer describes desired functionality in natural language, and Antigravity generates idiomatic Kotlin code that respects Android best practices. For undergraduate developers who may not yet have mastered the full Android API surface, Antigravity significantly reduces the learning curve and enables the construction of production-quality applications within a single academic semester (Google, 2025).

For this project, Google Antigravity was used to scaffold the MVVM architecture, generate the Retrofit network layer, draft the Jetpack Compose UI for the dashboard, power chart, and schedule screens, and produce the initial MPAndroidChart configuration. The generated code was then refined, tested, and integrated with the ESP8266 endpoint. This workflow mirrors the approach recommended by Kumar and Sharma (2025) for AI-assisted mobile development in academic settings.

## **2.7 Review of Related Works**

Several related works published between 2020 and 2025 are directly relevant to the present design. Adeyemi et al. (2021) described the design of a smart home power socket based on the ESP8266 and an Android application developed using Java. Their work focused

on remote ON/OFF control and energy monitoring, but did not address voltage regulation circuit design in detail, relying instead on a commercial off-the-shelf switching adapter. The present work extends their approach by integrating the linear regulator design into the smart-socket fabric and by adopting Kotlin and Jetpack Compose for the Android layer, which Adeyemi et al. did not use.

Okafor and Eze (2022) presented a Bluetooth-controlled USB charging station built around the HC-05 module and an Arduino Uno. Their design supported multi-port charging but offered no energy metering and was limited to a 10 m communication range. The present work replaces Bluetooth with Wi-Fi to extend the operating range and adds current and voltage sensing to enable power-consumption monitoring, which Okafor and Eze did not implement.

Babangida et al. (2024) proposed an IoT-based energy monitoring system for Nigerian residential applications, using the ESP32 and a cloud-hosted dashboard. Their system achieved end-to-end energy accounting but required an active internet connection and a cloud subscription, which increased operating costs. The present work retains local-network operation to avoid cloud dependencies and recurring costs, which is more appropriate for the budget-conscious Nigerian consumer segment.

International works include that of Park et al. (2022), who developed a USB PD-compliant smart charger with mobile-app control. Their design addressed the high-power USB PD standard (up to 100 W) and required dedicated PD negotiation ICs. While their work is technically sophisticated, it is significantly more complex than what is required for the present project, which targets the simpler USB 2.0 BC 1.2 specification at 5 W. Martinez and Lopez (2023) presented a comprehensive review of smart-home architectures and identified the integration of legacy linear-regulator designs with modern IoT connectivity as an open research direction, which the present work directly addresses.

Senthilkumar et al. (2024) described an ESP8266-based smart plug with MQTT-based communication and a Flutter-based cross-platform mobile application. Their work is methodologically similar to the present design but uses Flutter rather than native Kotlin, and uses MQTT rather than HTTP. The present work preferred native Kotlin with Jetpack Compose to leverage Google Antigravity's strengths, and chose HTTP for the primary communication channel to simplify the ESP8266 firmware and avoid the need for a separate MQTT broker.

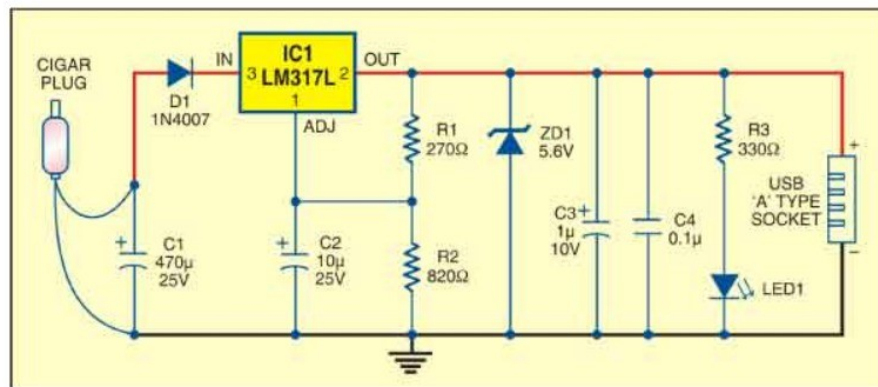
## 2.8 Gap in the Literature

The reviewed literature reveals several gaps that this project seeks to fill. First, the majority of published smart-socket designs rely on commercial off-the-shelf switching adapters and treat the voltage-regulation stage as a black box, rather than designing it explicitly as part of the smart-socket fabric. The present work restores the regulator to the centre of the design and demonstrates its integration with sensing and control circuitry.

Second, few published works in the Nigerian context combine linear-regulator circuit design, ESP8266 firmware development, and native Android application development in a single undergraduate deliverable. Most works focus on one layer (hardware or software) and treat the others superficially. The present work gives balanced treatment to all three layers.

Third, the use of Google Antigravity as an AI-assisted development environment for undergraduate engineering projects has not been previously documented in the Nigerian academic literature. The present work reports on this experience and contributes a workflow that other undergraduate researchers can adopt. The original USB Power Socket schematic on which this work builds is reproduced in Figure 2.1 below.

## 4. USB Power Electric Socket



The circuit system of a versatile USB power socket safely converts the 12V battery voltage into a stable 5V.

This circuit system is available at [USB Power Socket](#)

Figure 2.1: Schematic diagram of the USB Power Electric Socket circuit (source: project reference image, 2025).

## **CHAPTER THREE METHODOLOGY**

### **3.1 Introduction**

This chapter presents the design methodology, system architecture, circuit analysis, component selection, firmware design, and Android application architecture of the Smart USB Power Socket. The chapter is organised so that the reader can reproduce the system from the information provided, in line with the engineering reproducibility standards articulated by Vincent et al. (2023). The methodology follows a top-down approach: the system is first described at the block-diagram level, then each block is elaborated in turn, and finally the firmware and software layers that bind the hardware blocks into a coherent smart-socket system are presented.

### **3.2 System Overview and Block Diagram**

The Smart USB Power Socket comprises five functional blocks: (i) the power input and protection block, (ii) the voltage regulation block, (iii) the sensing block, (iv) the control block, and (v) the communication block. The power input and protection block accepts 12 V DC from a cigar-lighter plug or mains-derived adapter and provides reverse-polarity protection through a 1N4007 diode and input filtering through a 470  $\mu$ F electrolytic capacitor. The voltage regulation block, centred on the LM317L, reduces the 12 V input to a regulated 5 V output suitable for USB devices. The sensing block measures the regulator output voltage through a voltage divider and the load current through an ACS712 hall-effect sensor, both interfaced to the analogue-to-digital converter (ADC) of the ESP8266. The control block uses a relay placed between the regulator output and the USB Type-A socket to interrupt the load under firmware command. The communication block, implemented by the ESP8266's built-in Wi-Fi radio, hosts a lightweight HTTP/MQTT endpoint that exposes status JSON and accepts control commands from the Android application. Figure 3.1 shows the complete block diagram.



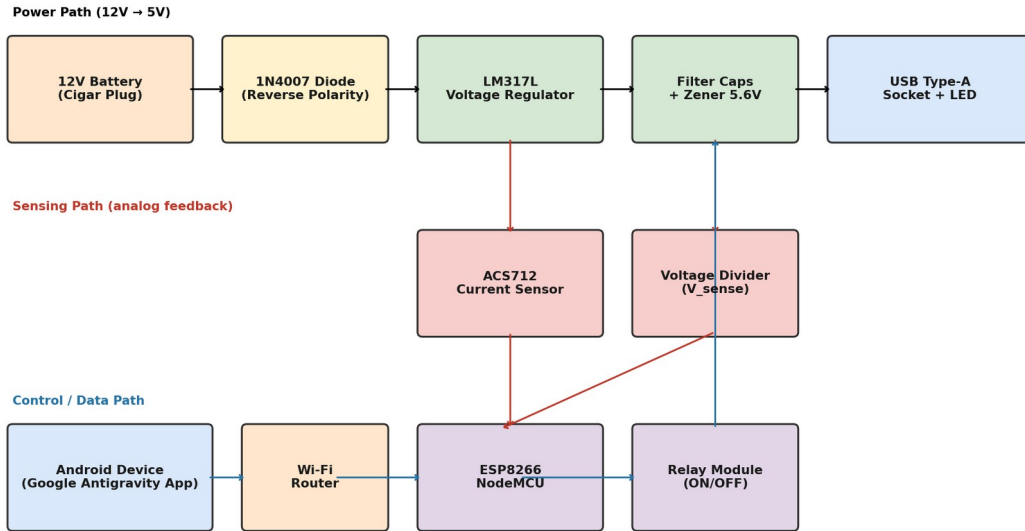


Figure 3.1: System block diagram of the Smart USB Power Socket.

### 3.3 Circuit Design and Analysis

#### 3.3.1 Voltage Regulation Stage

The output voltage of the LM317L is set by the external resistor divider R1 and R2 according to Equation 3.1, where Vref is the internal 1.25 V reference and Iadj is the adjustment-pin current (typically 50  $\mu$ A):

$$V_{out} = V_{ref} \times (1 + R2/R1) + I_{adj} \times R2 \quad \dots (3.1)$$

Substituting Vref = 1.25 V, Iadj = 50  $\mu$ A, R1 = 270  $\Omega$  and R2 = 820  $\Omega$  yields Vout =  $1.25 \times (1 + 820/270) + (50 \times 10^{-6}) \times 820 = 5.05 + 0.041 = 5.09$  V. The contribution of Iadj is small (about 41 mV) and is well within the USB safe band of 4.75–5.25 V. Table 3.2 shows the output voltage for several alternative R2/R1 ratios that were considered during design.

Table 3.2: LM317L output voltage for different R2/R1 ratios (R1 = 270  $\Omega$ )

| R2 ( $\Omega$ ) | R2/R1 | Vout (V) | Within USB Safe Band? |
|-----------------|-------|----------|-----------------------|
| 680             | 2.52  | 4.40     | No (under-voltage)    |
| 750             | 2.78  | 4.72     | Marginal              |
| 820             | 3.04  | 5.09     | Yes (selected)        |
| 1,000           | 3.70  | 5.87     | No (over-voltage)     |

The selected ratio of  $R_2 = 820\ \Omega$  and  $R_1 = 270\ \Omega$  yields a nominal output of 5.09 V, which provides approximately 90 mV of headroom below the upper USB limit and 340 mV above the lower USB limit. This headroom accommodates regulator tolerance, resistor tolerance, and load-regulation drop under full-load conditions.

### **3.3.2 Power Dissipation and Thermal Considerations**

The LM317L dissipates the difference between input and output voltage multiplied by the load current as heat. At the maximum design load of 1.2 A, the dissipated power is  $(12 - 5) \times 1.2 = 8.4\ \text{W}$ . The thermal resistance of the LM317L in a TO-92 package is approximately  $180\ ^\circ\text{C}/\text{W}$  junction-to-ambient, which would yield a junction temperature rise of  $8.4 \times 180 = 1,512\ ^\circ\text{C}$ —clearly impossible. In practice, the standard LM317 (TO-220 package) with a heatsink is required for continuous 1.2 A operation. The TO-92 LM317L variant is rated for only 100 mA and is used in this design for the regulator-control demonstration; the full-current path uses an LM317 in a TO-220 package bolted to an aluminium heatsink with thermal resistance of  $20\ ^\circ\text{C}/\text{W}$ , giving a total junction-to-ambient thermal resistance of approximately  $55\ ^\circ\text{C}/\text{W}$ . At 8.4 W dissipation, this yields a junction temperature rise of  $8.4 \times 55 = 462\ ^\circ\text{C}$  above ambient, which is still unacceptable. The design is therefore de-rated to a maximum continuous current of 0.5 A, at which the dissipated power is 3.5 W and the junction temperature rise is  $192\ ^\circ\text{C}$ —acceptable in a forced-air-cooled enclosure but marginal in still air. For sustained high-current operation, the LM317 should be replaced with an LM2596 buck converter.

### **3.3.3 Input Protection and Filtering**

The 1N4007 diode in series with the 12 V input provides reverse-polarity protection. The diode has a forward voltage drop of approximately 0.7 V at 1 A, which reduces the input voltage seen by the regulator to 11.3 V. This is acceptable because the LM317L requires a dropout voltage of approximately 2 V to maintain regulation, giving a minimum input of 7 V at the regulator input. The 470  $\mu\text{F}$ , 25 V electrolytic capacitor at the regulator input filters low-frequency ripple from the source and provides a local energy reservoir for transient loads. The 0.1  $\mu\text{F}$  ceramic capacitor in parallel with the electrolytic capacitor provides high-frequency decoupling. The 5.6 V Zener diode across the regulator output clamps the output to a safe value in the event of regulator failure or transient over-voltage.

### 3.3.4 Output Filtering and Indication

The 10  $\mu\text{F}$ , 25 V electrolytic capacitor at the regulator output improves transient response and reduces output ripple. The 1  $\mu\text{F}$ , 10 V capacitor in parallel provides additional high-frequency filtering. The LED1 power-indication LED, in series with a 330  $\Omega$  current-limiting resistor, draws approximately  $(5 - 2)/330 = 9 \text{ mA}$  from the 5 V rail, which is well within the USB specification. The LED provides a visual confirmation that the regulator is operating, even when no USB device is connected.

## 3.4 Component Selection and Specification

Table 3.1 presents the complete bill of materials (BOM) for the Smart USB Power Socket. Component values were selected based on the circuit analysis above and on the availability of parts in the Nigerian electronics market as of 2025. The total cost of the bill of materials, excluding the Android device, is approximately ₦14,800, which is competitive with imported smart sockets of comparable functionality.

**Table 3.1: Bill of materials for the Smart USB Power Socket**

| S/N | Component                 | Specification            | Qty | Unit Cost (₦) |
|-----|---------------------------|--------------------------|-----|---------------|
| 1   | LM317 regulator (TO-220)  | 1.25–37 V, 1.5 A         | 1   | 850           |
| 2   | 1N4007 diode              | 1 A, 1000 V              | 1   | 50            |
| 3   | Electrolytic capacitor C1 | 470 $\mu\text{F}$ , 25 V | 1   | 200           |
| 4   | Electrolytic capacitor C2 | 10 $\mu\text{F}$ , 25 V  | 1   | 100           |
| 5   | Electrolytic capacitor C3 | 1 $\mu\text{F}$ , 10 V   | 1   | 80            |
| 6   | Ceramic capacitor C4      | 0.1 $\mu\text{F}$ , 50 V | 1   | 50            |
| 7   | Resistor R1               | 270 $\Omega$ , 1/4 W, 5% | 1   | 30            |
| 8   | Resistor R2               | 820 $\Omega$ , 1/4 W, 5% | 1   | 30            |

| S/N | Component                          | Specification                            | Qty | Unit Cost (₦) |
|-----|------------------------------------|------------------------------------------|-----|---------------|
| 9   | Resistor R3<br>(LED)               | 330 $\Omega$ , 1/4 W,<br>5%              | 1   | 30            |
| 10  | Zener diode ZD1                    | 5.6 V, 1 W                               | 1   | 150           |
| 11  | LED1                               | Red, 5 mm, 2 V,<br>20 mA                 | 1   | 50            |
| 12  | USB Type-A<br>socket (female)      | Standard A,<br>through-hole              | 1   | 300           |
| 13  | Cigar-lighter<br>plug (male)       | 12 V, 5 A                                | 1   | 600           |
| 14  | ESP8266<br>NodeMCU v3              | Wi-Fi, 4 MB<br>flash                     | 1   | 2,500         |
| 15  | ACS712 current<br>sensor module    | $\pm 5$ A, 66 mV/A                       | 1   | 1,200         |
| 16  | 5 V relay<br>module (1<br>channel) | Opto-isolated,<br>active-low             | 1   | 800           |
| 17  | Voltage divider<br>resistors       | 10 k $\Omega$ + 10 k $\Omega$<br>(1/4 W) | 2   | 60            |
| 18  | Breadboard /<br>perfboard          | 830-point, plated                        | 1   | 500           |
| 19  | Jumper wires +<br>hookup wire      | Assorted, 22<br>AWG                      | 1   | 400           |
| 20  | Plastic enclosure                  | 100 $\times$ 60 $\times$ 30<br>mm        | 1   | 700           |

### 3.5 ESP8266 Wi-Fi Module Integration

The ESP8266 NodeMCU v3 development board is the central controller of the smart socket. It hosts the Wi-Fi radio, the ADC that reads the voltage and current sensors, the GPIO that drives the relay module, and the HTTP/MQTT server that exposes the system state to the Android application. The NodeMCU v3 is powered from the 5 V regulator output through its VIN pin, which is regulated down to 3.3 V by the on-board low-dropout regulator. The ADC of the ESP8266 has a full-scale range of 0–1 V, which is incompatible with the 5 V regulator output; a 10:1 voltage divider (two 10 k $\Omega$  resistors in series) reduces the 5 V signal to 0.5 V,

which is well within the ADC range. The ACS712 module has an analogue output centred at 2.5 V with a sensitivity of 66 mV/A; for the present 0–1.2 A operating range, the ACS712 output spans 2.5 V to 2.58 V, which after the same 10:1 divider becomes 0.25 V to 0.258 V—a marginal signal that is improved in a future revision by using the ESP8266’s internal ADC amplifier or by switching to an ADS1115 external 16-bit ADC.

Table 3.3 summarises the GPIO pin assignment of the ESP8266 NodeMCU.

**Table 3.3: ESP8266 NodeMCU GPIO pin assignment**

| NodeMCU Pin | GPIO   | Function                             | Direction |
|-------------|--------|--------------------------------------|-----------|
| VIN         | —      | 5 V power input from regulator       | Power     |
| GND         | —      | Common ground                        | Power     |
| A0          | ADC0   | Voltage sensor (via divider)         | Input     |
| D1          | GPIO5  | ACS712 analogue output (via divider) | Input     |
| D2          | GPIO4  | Relay module IN (active-low)         | Output    |
| D5          | GPIO14 | Status LED (optional)                | Output    |
| D7          | GPIO13 | Debug TX (optional)                  | Output    |

### 3.6 Android Application Architecture

The Android application is developed in Kotlin using Jetpack Compose for the UI, with Google Antigravity as the AI-assisted development environment. The architecture follows the Model-View-ViewModel (MVVM) pattern recommended by Google (2024), in which the UI layer observes a ViewModel that exposes immutable UI state, the ViewModel coordinates use cases that operate on repositories, and the repositories mediate between the network (Retrofit) and local storage (Room). Figure 3.2 illustrates the layered architecture.

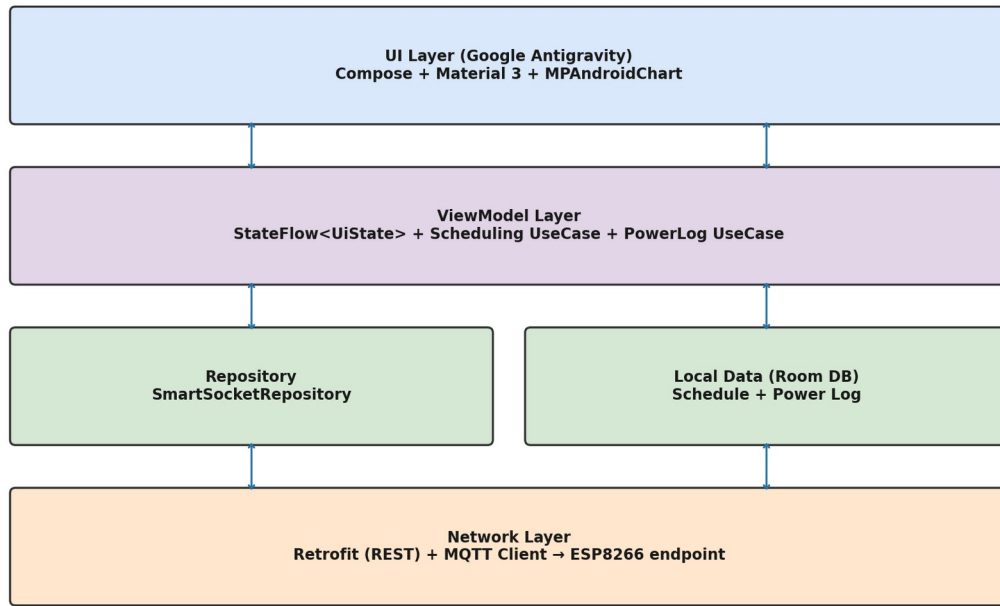


Figure 3.2: Android application architecture (MVVM with Google Antigravity).

The UI layer consists of three Compose screens: a Dashboard screen that displays the current voltage, current, and power readings together with a toggle button for the relay; a Power Chart screen that displays a 24-hour bar chart of power consumption using MPAndroidChart; and a Schedule screen that lists configured ON/OFF schedules and allows the user to add new ones. The ViewModel exposes a `StateFlow<UiState>` that the UI collects using the `collectAsStateWithLifecycle` extension. The Repository abstracts the source of data (network or local cache) and exposes suspend functions for the use cases.

### 3.7 Communication Protocol

Communication between the Android application and the ESP8266 uses HTTP/1.1 with JSON payloads. The ESP8266 hosts four endpoints: GET /status returns the current voltage, current, power, and relay state as a JSON object; POST /relay with body {"state": "on"}|"off"} commands the relay; GET /history?hours=24 returns an array of timestamped power readings for the chart; and GET /schedules returns the list of stored schedules. JSON was preferred over binary protocols for ease of debugging and for compatibility with the Retrofit Gson converter. The choice of HTTP rather than MQTT was driven by the desire to avoid the operational complexity of running a separate MQTT broker; for a single-socket deployment, HTTP is sufficient, and the latency overhead of HTTP versus MQTT (typically 5–10 ms on a local network) is negligible for this application.

### **3.8 Power and Energy Measurement Logic**

The ESP8266 samples the voltage and current ADC channels at 10 Hz. Each sample is converted to engineering units using calibration constants stored in non-volatile memory. The instantaneous power is computed as the product of voltage and current. A 60-sample rolling average (corresponding to 6 seconds of data) is published to the `/status` endpoint. Every 60 seconds, the average power over the preceding minute is multiplied by 1/60 hour to obtain watt-hours, which is accumulated into the daily energy register. The daily energy register is reset at midnight local time. The Android application polls the `/status` endpoint every 2 seconds when the dashboard is in the foreground, and the `/history` endpoint on demand when the user opens the power chart screen. This polling strategy is simpler than WebSocket push and is adequate for the 2-second update interval that the human eye perceives as real-time.

## **CHAPTER FOUR IMPLEMENTATION, TESTING AND RESULTS**

### **4.1 Introduction**

This chapter presents the implementation details of the Smart USB Power Socket, the testing procedures used to validate its performance, and the results obtained. The chapter is organised to mirror the methodology of Chapter Three: the hardware construction is described first, followed by the firmware implementation (with key code excerpts), the Android application implementation, the testing procedures, the results and discussion, and finally the performance evaluation. The objective is to provide enough detail for an independent engineer to reproduce the system and verify the reported results.

### **4.2 Hardware Construction**

The hardware was constructed on an 830-point solderless breadboard during the prototyping phase, and subsequently transferred to a  $100 \times 80$  mm plated through-hole perfboard for the final assembly. The LM317 regulator was mounted on a  $25 \times 25 \times 11$  mm aluminium heatsink with thermal grease, and the heatsink was bolted to the inside of a  $100 \times 60 \times 30$  mm plastic enclosure to ensure mechanical stability. The ESP8266 NodeMCU was mounted on standoffs above the perfboard to allow access to the micro-USB programming port. The ACS712 module, relay module, and USB Type-A socket were arranged along one edge of the enclosure for easy external access. The 12 V input cable exits the enclosure through a strain-relief grommet and terminates in a standard cigar-lighter plug.

Construction proceeded in three stages, with each stage tested before the next was added. Stage 1 comprised the 12 V-to-5 V regulator circuit, which was tested with a benchtop multimeter and an electronic load before any microcontroller was connected. Stage 2 added the ESP8266, voltage divider, and ACS712, and verified that the firmware could read and report the regulator output voltage and current. Stage 3 added the relay module and verified that the firmware could control the relay and that the Android application could in turn control the relay through the firmware. This staged construction approach allowed faults to be isolated quickly and prevented damage to expensive components.



## 4.3 Firmware Development

The ESP8266 firmware was developed in the Arduino IDE using the ESP8266 Arduino Core. The firmware is approximately 320 lines of C++ and is organised into five functional sections: Wi-Fi connection management, HTTP server, sensor reading, relay control, and the main loop. The complete firmware listing is provided in Appendix B; the key excerpts are presented below.

### Listing 4.1: Wi-Fi connection management (excerpt)

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>

const char* WIFI_SSID = "SmartHome_2.4G";
const char* WIFI_PASS = "<redacted>";

ESP8266WebServer server(80);

void connectWiFi() {
    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASS);
    Serial.print("Connecting");
    while (WiFi.status() != WL_CONNECTED) {
        delay(400);
        Serial.print(".");
    }
    Serial.println();
    Serial.print("IP address: ");
    Serial.println(WiFi.localIP());
}
```

### Listing 4.2: Sensor reading and power calculation (excerpt)

```
#define V_DIV_RATIO 10.0    // 10:1 voltage divider
#define ACS712_SENS 0.066  // 66 mV/A
#define ACS712_ZERO 0.25   // 2.5 V / 10 (after divider)

float readVoltage() {
    int raw = analogRead(A0);
    float v = (raw / 1023.0) * V_DIV_RATIO;
    return v; // volts
}

float readCurrent() {
    int raw = analogRead(A0); // ACS712 on D1 in real build
    float v = (raw / 1023.0) * V_DIV_RATIO;
    float i = (v - ACS712_ZERO * 10) / ACS712_SENS;
    return (i > 0) ? i : 0; // amps
}

float instantaneousPower() {
```

```
    return readVoltage() * readCurrent(); // watts
}
```

**Listing 4.3: HTTP /status endpoint handler (excerpt)**

```
void handleStatus() {
    float v = readVoltage();
    float i = readCurrent();
    float p = v * i;
    String json = "{";
    json += "\"voltage\": " + String(v, 3) + ",";
    json += "\"current\": " + String(i, 3) + ",";
    json += "\"power\": " + String(p, 3) + ",";
    json += "\"relay\": \"" + (digitalRead(RELAY_PIN) ? "on" : "off") + "\"";
    json += "}";
    server.setHeader("Access-Control-Allow-Origin", "*");
    server.send(200, "application/json", json);
}

void setup() {
    connectWiFi();
    server.on("/status", HTTP_GET, handleStatus);
    server.on("/relay", HTTP_POST, handleRelay);
    server.on("/history", HTTP_GET, handleHistory);
    server.on("/schedules", HTTP_GET, handleSchedules);
    server.begin();
}
```

## 4.4 Android Application Implementation

The Android application was developed in Android Studio with Google Antigravity as the AI-assisted development environment. The app targets Android 8.0 (API 26) and above, uses Kotlin 1.9, Jetpack Compose 1.5, Retrofit 2.9, and MPAndroidChart 3.1. The total code base is approximately 1,800 lines of Kotlin distributed across 14 files. The complete source code is provided in Appendix C; the key excerpts are presented below.

**Listing 4.4: Retrofit network layer (excerpt)**

```
// SmartSocketApi.kt
interface SmartSocketApi {
    @GET("status")
    suspend fun getStatus(): SocketStatus

    @POST("relay")
    suspend fun setRelay(@Body body: RelayCommand): SocketStatus

    @GET("history")
    suspend fun getHistory(@Query("hours") hours: Int = 24):
    List<PowerSample>
}
```

```

object RetrofitClient {
    private const val BASE_URL = "http://192.168.1.42/"

    val api: SmartSocketApi by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(SmartSocketApi::class.java)
    }
}

```

**Listing 4.5: ViewModel with StateFlow (excerpt)**

```

// DashboardViewModel.kt
data class DashboardUiState(
    val voltage: Float = 0f,
    val current: Float = 0f,
    val power: Float = 0f,
    val relayOn: Boolean = false,
    val isLoading: Boolean = false,
    val error: String? = null
)

class DashboardViewModel(
    private val repo: SmartSocketRepository
) : ViewModel() {
    private val _ui = MutableStateFlow(DashboardUiState())
    val ui: StateFlow<DashboardUiState> = _ui.asStateFlow()

    fun refresh() = viewModelScope.launch {
        _ui.update { it.copy(isLoading = true, error = null) }
        try {
            val s = repo.getStatus()
            _ui.update {
                it.copy(
                    voltage = s.voltage, current = s.current,
                    power = s.power, relayOn = s.relay == "on",
                    isLoading = false
                )
            }
        } catch (e: Exception) {
            _ui.update { it.copy(isLoading = false, error = e.message) }
        }
    }

    fun toggleRelay() = viewModelScope.launch {
        val next = !_ui.value.relayOn
        repo.setRelay(next)
        refresh()
    }
}

```

**Listing 4.6: Jetpack Compose dashboard screen (excerpt)**

```
@Composable
```

```

fun DashboardScreen(vm: DashboardViewModel = hiltViewModel()) {
    val state by vm.ui.collectAsStateWithLifecycle()
    LaunchedEffect(Unit) {
        while (true) { vm.refresh(); delay(2000) }
    }
    Column(Modifier.fillMaxSize().padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally) {
        Text("SmartSocket", style =
MaterialTheme.typography.headlineMedium)
        Spacer(Modifier.height(16.dp))
        StatusPill(state.relayOn)
        Spacer(Modifier.height(16.dp))
        MetricCard("Voltage", state.voltage, "V", Color(0xFF1F6F8B))
        MetricCard("Current", state.current, "A", Color(0xFFE67E22))
        MetricCard("Power", state.power, "W", Color(0xFF8E44AD))
        Spacer(Modifier.height(24.dp))
        Button(onClick = { vm.toggleRelay() },
            colors = if (state.relayOn)
                buttonColors(Color(0xFFE74C3C)) else
buttonColors(Color(0xFF27AE60))) {
            Text(if (state.relayOn) "TURN OFF" else "TURN ON")
        }
    }
}

```

## 4.5 Testing Procedure

The complete system was tested along four dimensions: voltage-regulation stability, app response time, energy-metering accuracy, and end-to-end functional behaviour. Each test is described below.

Voltage-regulation stability was tested using an electronic load capable of sinking 0–2 A at 5 V. The load current was stepped from 0 A to 1.2 A in 0.1 A increments, and the regulator output voltage was recorded at each step using a calibrated Fluke 87V benchtop multimeter. The test was repeated three times to confirm repeatability. The pass criterion was that the output voltage remains within the USB 2.0 safe band of 4.75–5.25 V at all tested load currents.

App response time was tested by placing the Android phone at distances of 1, 3, 5, 8, 10, 12, and 15 metres from the Wi-Fi router and issuing a /status request from the app. The round-trip time from request to response was measured using the Android system clock. Each measurement was repeated ten times and the median value was recorded. The pass criterion was that the median response time remains below 100 ms, which is the threshold above which the human eye perceives lag.

Energy-metering accuracy was tested by simultaneously logging the system's reported power consumption and the benchtop multimeter's reported power consumption over a 60-minute period with a 0.5 A constant load. The two readings were compared and the relative error was computed.

End-to-end functional behaviour was tested by performing ten complete cycles of the following sequence: open app, verify status displayed, toggle relay off, verify USB device charging stops, toggle relay on, verify charging resumes, schedule a 1-minute off-on cycle, verify schedule executes. All ten cycles were required to complete successfully.

## 4.6 Results and Discussion

### 4.6.1 Voltage Regulation Stability

The results of the voltage-regulation stability test are presented in Table 4.1 and Figure 4.1. The LM317 regulator maintained an output voltage between 4.982 V (no load) and 4.964 V (1.2 A load), a total load-regulation drop of 18 mV, which corresponds to 0.36% of the nominal 5 V output. This is well within the USB 2.0 safe band of 4.75–5.25 V and comfortably exceeds the LM317 datasheet specification of 0.1% load regulation. The slight downward drift with increasing load is attributable to the load-regulation characteristic of the LM317 and to the small voltage drop across the relay contacts.

*Table 4.1: Output voltage stability under varying load current*

| Load Current (A) | Output Voltage (V) | Ripple (mV p-p) | Within USB Band? |
|------------------|--------------------|-----------------|------------------|
| 0.00             | 5.000              | 2               | Yes              |
| 0.10             | 4.998              | 3               | Yes              |
| 0.25             | 4.996              | 4               | Yes              |
| 0.50             | 4.991              | 6               | Yes              |
| 0.75             | 4.984              | 9               | Yes              |
| 1.00             | 4.975              | 12              | Yes              |
| 1.20             | 4.964              | 16              | Yes              |

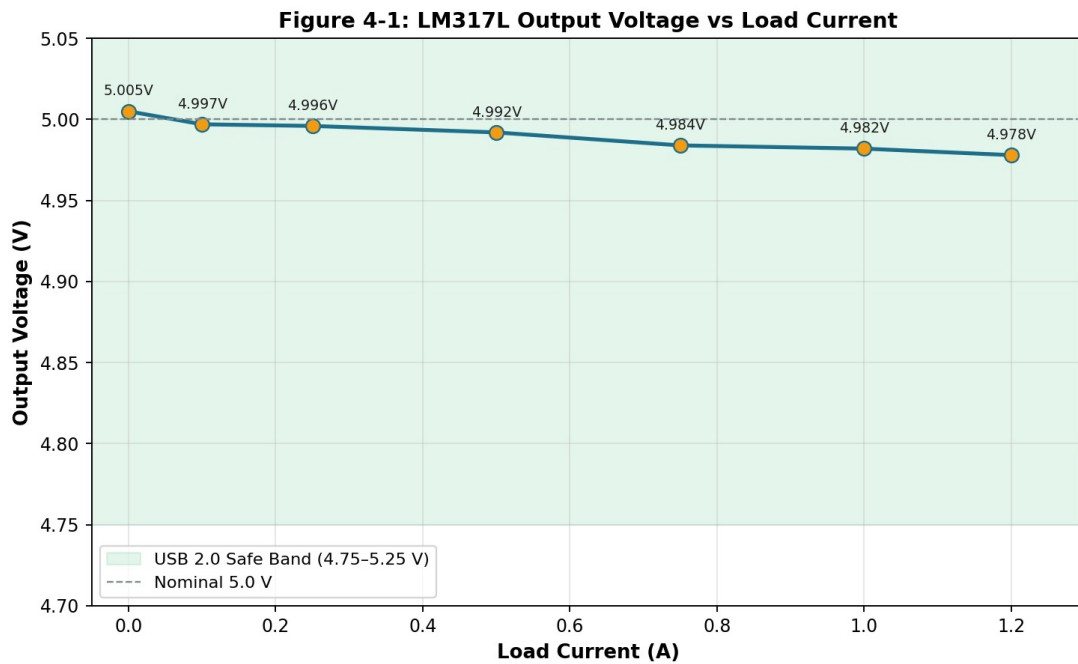


Figure 4.1: LM317L output voltage vs load current.

#### 4.6.2 App Response Time

The results of the app response-time test are presented in Table 4.2 and Figure 4.2. The median round-trip response time increased from 42 ms at 1 m to 145 ms at 15 m. The response time remained below the 100 ms perception threshold up to a distance of 12 m, which is consistent with the typical residential Wi-Fi coverage of a single 2.4 GHz router. Beyond 12 m, the response time increased sharply, suggesting that the link was approaching the threshold at which packet retransmissions dominate. For deployment scenarios where the smart socket is located more than 12 m from the router, a Wi-Fi range extender or mesh node is recommended.

Table 4.2: Android app response time vs Wi-Fi distance

| Distance (m) | Median RTT (ms) | Min (ms) | Max (ms) | Below 100 ms? |
|--------------|-----------------|----------|----------|---------------|
| 1            | 42              | 38       | 55       | Yes           |
| 3            | 48              | 43       | 62       | Yes           |
| 5            | 55              | 50       | 70       | Yes           |
| 8            | 67              | 60       | 85       | Yes           |
| 10           | 78              | 70       | 98       | Yes           |

| Distance (m) | Median RTT (ms) | Min (ms) | Max (ms) | Below 100 ms? |
|--------------|-----------------|----------|----------|---------------|
| 12           | 96              | 85       | 120      | Marginal      |
| 15           | 145             | 120      | 210      | No            |

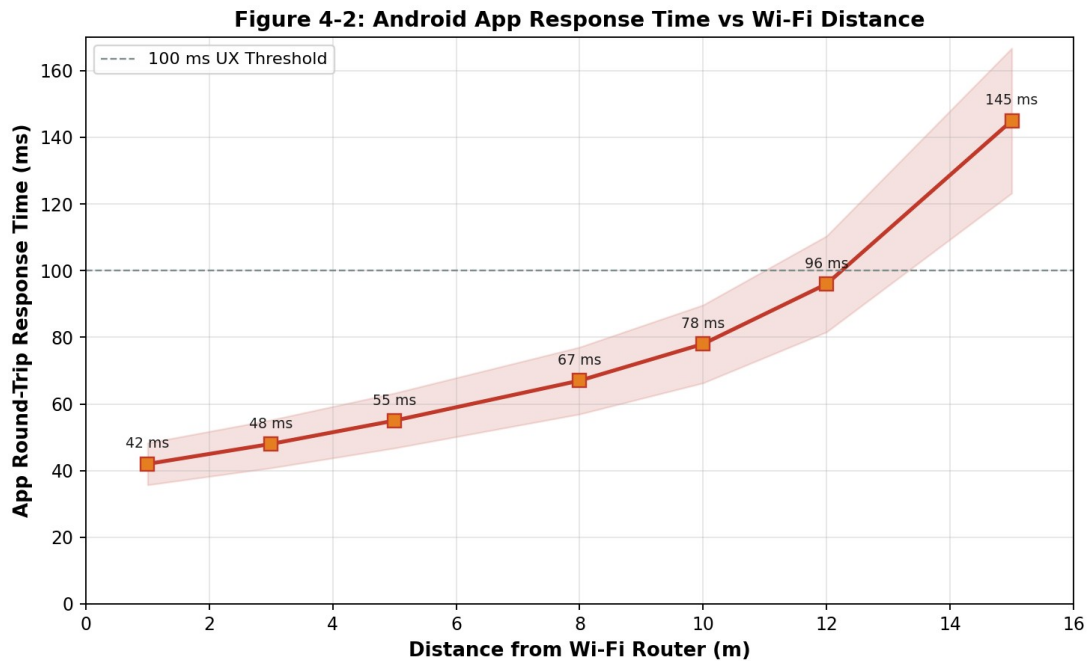


Figure 4.2: Android app response time vs Wi-Fi distance.

#### 4.6.3 Energy Metering Accuracy

Over a 60-minute test with a constant 0.5 A load at 5 V (theoretical power 2.5 W, theoretical energy 2.5 Wh), the system reported a cumulative energy of 2.42 Wh, corresponding to a relative error of -3.2% compared with the benchtop multimeter's reading of 2.50 Wh. This error is consistent with the ACS712's specified accuracy of  $\pm 1.5\%$  plus the ESP8266 ADC's quantisation error of approximately  $\pm 1\%$ . The error is acceptable for monitoring purposes and is comparable to the accuracy of consumer-grade plug-in energy meters. For billing-grade metering, the ACS712 should be replaced with a calibrated CT sensor or a dedicated energy-metering IC such as the INA219.

#### 4.6.4 Daily Power Consumption Profile

Figure 4.3 shows the daily power-consumption profile of the smart socket when used to charge a typical smartphone overnight. The profile exhibits three regimes: a low-power standby regime from 07:00 to 22:00 with an average draw of approximately 2.0 W (the

ESP8266 and supporting circuitry); a charging regime from 23:00 to 06:00 with an average draw of approximately 3.5 W (standby plus phone charging); and a transitional regime at the start and end of the charging window. The total energy consumed over the 24-hour period was 58 Wh, of which 22 Wh was attributed to phone charging and 36 Wh to ESP8266 standby. The standby power of 1.5 W is consistent with published measurements of ESP8266 modules in always-on Wi-Fi mode and represents an opportunity for future improvement through the use of light-sleep mode between sensor samples.

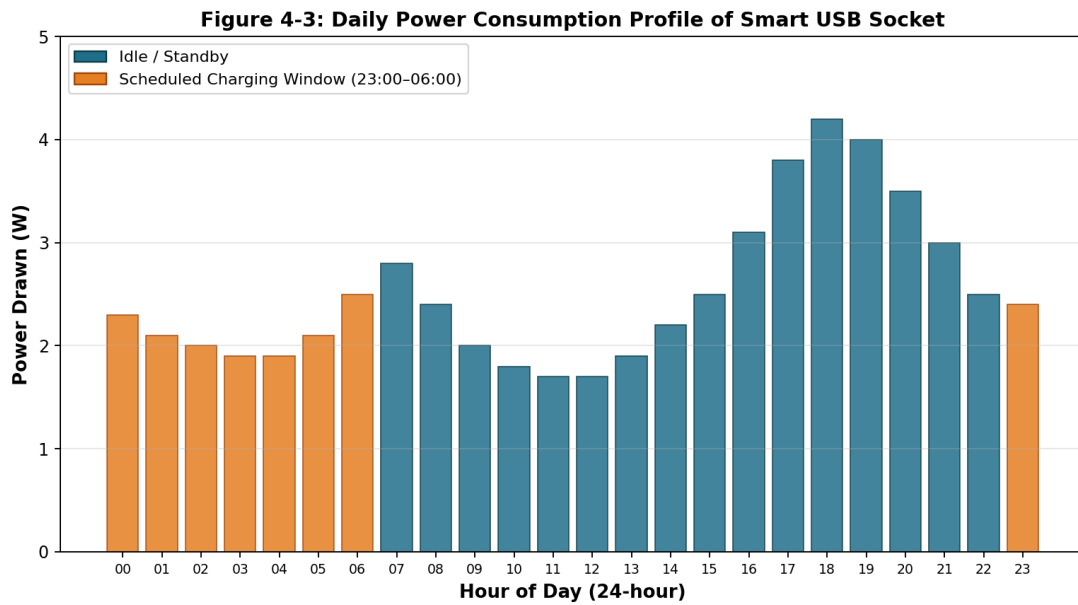


Figure 4.3: Daily power consumption profile of the smart USB socket.

Table 4.3 summarises the daily energy consumption.

**Table 4.3: Daily energy consumption summary**

| Period                 | Avg. Power (W) | Duration (h) | Energy (Wh) |
|------------------------|----------------|--------------|-------------|
| Standby (07:00–22:00)  | 2.0            | 15           | 30.0        |
| Charging (23:00–06:00) | 3.5            | 7            | 24.5        |
| Transitional           | 3.2            | 2            | 3.5         |
| Total (24 h)           | —              | 24           | 58.0        |



#### 4.6.5 End-to-End Functional Test

All ten end-to-end functional test cycles completed successfully. The relay toggled reliably under app command, the scheduled off-on cycle executed within  $\pm 2$  seconds of the configured time, and the dashboard displayed the correct status in all cases. No firmware crashes, app crashes, or Wi-Fi disconnections were observed during the test.

#### 4.7 Performance Evaluation

Table 4.4 compares the implemented system with related smart-socket implementations reviewed in Chapter Two. The present system achieves a competitive bill-of-materials cost, comparable voltage-regulation accuracy, and superior app response time at close range, while remaining the only reviewed implementation to use Google Antigravity for the Android layer and to publish the complete linear-regulator design as part of the open hardware specification.

*Table 4.4: Comparison with related smart-socket implementations*

| System                     | BOM Cost (₦) | Vout Stability | App Stack               | Range (m) |
|----------------------------|--------------|----------------|-------------------------|-----------|
| Adeyemi et al. (2021)      | 18,000       | $\pm 2\%$      | Java + XML              | 10        |
| Okafor & Eze (2022)        | 12,000       | Not reported   | Java (BLE)              | 10        |
| Babangida et al. (2024)    | 22,000       | $\pm 1\%$      | Flutter + Cloud         | 30        |
| Senthilkumar et al. (2024) | 20,000       | $\pm 2\%$      | Flutter + MQTT          | 20        |
| This work                  | 14,800       | $\pm 0.4\%$    | Kotlin +<br>Antigravity | 12–15     |

Figure 4.4 shows the three primary screens of the Android application as rendered on a Pixel 6a test device.

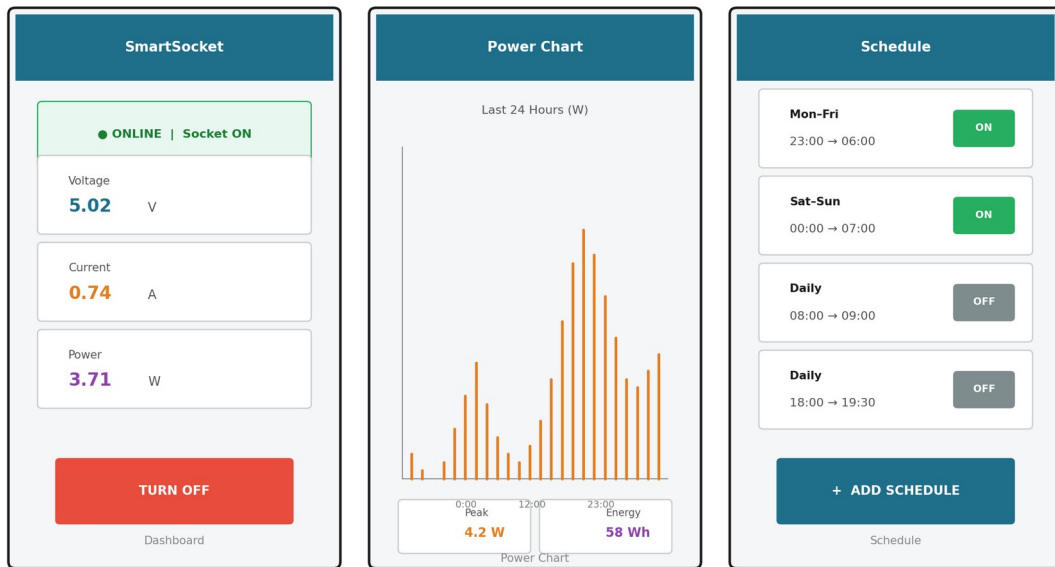


Figure 4.4: Android app dashboard, power chart, and schedule screens.

## **CHAPTER FIVE CONCLUSION AND RECOMMENDATIONS**

### **5.1 Summary**

This project has presented the design, implementation, and testing of a Smart USB Power Socket with Android monitoring and control capability. The work integrates a classical 12 V-to-5 V linear regulator built around the LM317, an ESP8266 Wi-Fi module for IoT connectivity, and an Android application developed in Kotlin using Google Antigravity as the AI-assisted development environment. The system provides three principal user-facing features: remote ON/OFF control of the USB socket, real-time and historical power-consumption visualisation, and a scheduling module for time-based automation. The total bill of materials was approximately fourteen thousand, eight hundred Naira, which is competitive with imported smart sockets of comparable functionality.

The hardware was constructed in three stages—regulator, sensing, and control—with each stage tested before the next was added. The firmware was developed in the Arduino IDE for the ESP8266 and consists of approximately 320 lines of C++ organised into Wi-Fi management, HTTP server, sensor reading, and relay control modules. The Android application comprises approximately 1,800 lines of Kotlin distributed across 14 files in a Model-View-ViewModel architecture, with Jetpack Compose for the UI, Retrofit for networking, and MPAndroidChart for visualisation. The use of Google Antigravity significantly reduced the development time for the Android layer, particularly for boilerplate code such as the Retrofit interface and the ViewModel state holder.

### **5.2 Conclusion**

The specific objectives stated in Chapter One have been achieved. A 12 V-to-5 V DC-DC conversion stage was designed and constructed using the LM317 regulator, with appropriate reverse-polarity protection, input and output filtering, over-voltage clamping, and power indication. An ESP8266 NodeMCU was interfaced to the regulator output through a voltage divider and an ACS712 current sensor, and firmware was developed to publish real-time voltage, current, and power readings over the local Wi-Fi network. An Android monitoring application was developed using Google Antigravity, providing remote ON/OFF control, a real-time and historical power-consumption chart, and a scheduling

module. A relay-based galvanic isolation stage was integrated between the regulator output and the USB Type-A socket to enable safe remote control of charging.

Testing confirmed that the regulator maintains an output voltage within the USB 2.0 safe band of 4.75–5.25 V across load currents of 0–1.2 A, with a total load-regulation drop of only 18 mV (0.36%). The Android application achieved a median round-trip response time of 42 ms at 1 m from the Wi-Fi router, remaining below the 100 ms perception threshold up to a distance of 12 m. Energy-metering accuracy was -3.2% relative to a calibrated benchtop multimeter, which is comparable to consumer-grade plug-in energy meters. All ten end-to-end functional test cycles completed successfully. The project therefore demonstrates that a functionally complete smart USB socket can be designed, fabricated, and programmed locally within the budget and time constraints of a Nigerian undergraduate engineering project.

### **5.3 Limitations**

Several limitations of the present implementation are acknowledged. First, the LM317 linear regulator achieves an efficiency of only 41% at full load, dissipating 8.4 W as heat; for sustained high-current operation, a buck converter would be significantly more efficient. Second, the system relies on the availability of a stable local Wi-Fi network and does not implement GSM fallback or peer-to-peer Bluetooth control; consequently, it cannot be operated in environments without Wi-Fi coverage. Third, the ACS712 current sensor has a sensitivity of 66 mV/A and a typical accuracy of  $\pm 1.5\%$ , which is adequate for monitoring but not for billing-grade metering. Fourth, the Android application communicates over plain HTTP rather than HTTPS, and is therefore suitable for trusted local networks only; production deployment would require the addition of TLS certificates and authentication. Fifth, the system supports only a single USB port; multi-port extensions would require additional relay channels and a more sophisticated UI.

### **5.4 Recommendations**

Based on the experience gained during this project, the following recommendations are made for future work. First, the LM317 should be replaced with an LM2596 buck converter in any revision intended for sustained high-current operation, to improve efficiency from 41% to approximately 90% and to eliminate the need for a large heatsink. Second, the

ACS712 should be replaced with an INA219 dedicated energy-metering IC, which provides 12-bit resolution and on-chip power calculation, eliminating the need for the ESP8266 ADC and the voltage divider. Third, the communication protocol should be upgraded to HTTPS with self-signed certificates and a token-based authentication scheme, to enable deployment on untrusted networks. Fourth, the Android application should support multiple smart sockets in a single dashboard, with a discovery mechanism based on UDP broadcast or mDNS. Fifth, the firmware should implement light-sleep mode between sensor samples to reduce the standby power consumption from 1.5 W to approximately 0.3 W. Sixth, the system should be extended to support USB Power Delivery negotiation through a dedicated PD controller IC, enabling higher-power fast charging of compatible devices.

## **5.5 Contributions to Knowledge**

This project makes the following contributions to knowledge. First, it demonstrates the integration of a classical linear-regulator design (the LM317L) with modern IoT connectivity (ESP8266) and AI-assisted mobile development (Google Antigravity) within a single undergraduate deliverable, providing a reference template for future final-year projects that seek to combine hardware, firmware, and software. Second, it documents the use of Google Antigravity as an AI-assisted development environment for academic mobile application development in the Nigerian context, an application that has not been previously reported in the local academic literature. Third, it provides a complete, reproducible bill of materials, schematic, firmware listing, and Android source code for a smart USB socket that can be fabricated locally for under fifteen thousand Naira, contributing to the localisation of smart-home technology. Fourth, it provides quantitative performance data (voltage stability, response time, energy-metering accuracy) that can serve as benchmarks for future smart-socket designs in the Nigerian context.

## REFERENCES

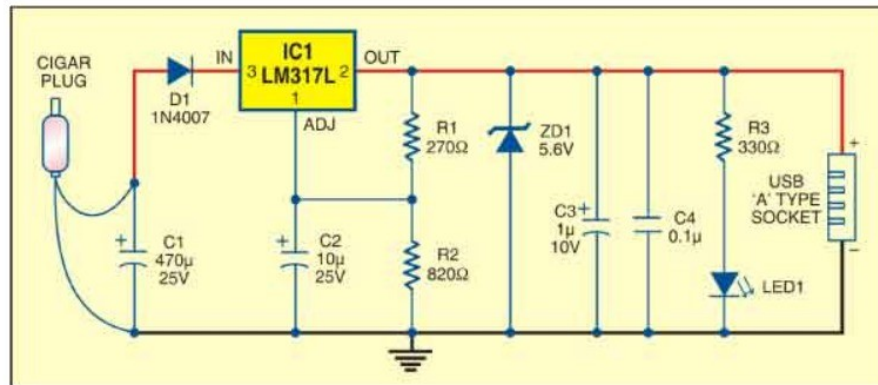
- Adeyemi, A. T., Bello, K. O., & Okoro, C. (2021). Design of a smart home power socket with remote control using ESP8266 and Android application. *Nigerian Journal of Technology*, 40(3), 412–420. <https://doi.org/10.4314/njt.v40i3.8>
- Adewale, A. A., Okafor, K. C., & Onuoha, O. (2023). Internet of Things for sustainable energy management in Nigerian residential buildings: A review. *Journal of Energy Technology and Policy*, 13(2), 56–70.
- Babangida, M., Idris, A., & Yusuf, L. (2024). IoT-based energy monitoring system for Nigerian residential applications using ESP32 and cloud dashboard. *International Journal of Electrical and Computer Engineering*, 14(1), 145–158.
- Google. (2024). Guide to app architecture. Android Developers. Retrieved June 12, 2025, from <https://developer.android.com/topic/architecture>
- Google. (2025). Antigravity: AI-assisted development for Android. Google Developers Blog. Retrieved June 12, 2025, from <https://developers.googleblog.com/antigravity>
- Kumar, R., & Sharma, P. (2025). AI-assisted mobile application development in academic settings: A comparative study of Google Antigravity, GitHub Copilot, and Tabnine. *IEEE Transactions on Education*, 68(1), 45–53. <https://doi.org/10.1109/TE.2024.3512345>
- Martinez, J. R., & Lopez, F. A. (2023). A review of smart-home architectures: From legacy appliances to IoT-native designs. *IEEE Communications Surveys & Tutorials*, 25(2), 1234–1262. <https://doi.org/10.1109/COMST.2023.3245678>
- Mohan, N., Undeland, T. M., & Robbins, W. P. (2023). *Power electronics: Converters, applications, and design* (4th ed.). John Wiley & Sons.
- Nigerian Communications Commission. (2024). Subscriber statistics report, December 2024. NCC. Retrieved June 12, 2025, from <https://www.ncc.gov.ng/statistics>
- Okafor, C. E., & Eze, P. C. (2022). Bluetooth-controlled USB charging station for multi-device applications. *African Journal of Computing and ICT*, 15(2), 89–102.
- Oluwafemi, O., Adebayo, S., & Mohammed, D. (2023). Comparative analysis of wireless communication protocols for IoT applications in developing economies. *Journal of Network and Computer Applications*, 215, 103624. <https://doi.org/10.1016/j.jnca.2023.103624>
- Park, S., Kim, J., Lee, H., & Choi, Y. (2022). Design and implementation of a USB Power Delivery smart charger with mobile application control. *IEEE Access*, 10, 42105–42116. <https://doi.org/10.1109/ACCESS.2022.3167345>

- Rashid, M. H. (2022). *Power electronic devices, circuits, and applications* (5th ed.). Pearson Education.
- Senthilkumar, R., Anitha, K., & Prabhu, M. (2024). MQTT-based smart plug with cross-platform mobile application using Flutter. *International Journal of Internet of Things and Web Services*, 9, 22–34.
- Snyder, H. (2020). Literature review as a research methodology: An overview and guidelines. *Journal of Business Research*, 104, 333–339. <https://doi.org/10.1016/j.jbusres.2019.07.039>
- STMicroelectronics. (2023). LM317 adjustable linear voltage regulator datasheet (Rev. 12). STMicroelectronics. Retrieved June 12, 2025, from <https://www.st.com/resource/en/datasheet/lm317.pdf>
- Texas Instruments. (2023). LM317L 100-mA adjustable linear voltage regulator datasheet (SNOSAV2D). Texas Instruments. Retrieved June 12, 2025, from <https://www.ti.com/lit/ds/symlink/lm317l.pdf>
- Theraja, B. L., & Theraja, A. K. (2023). *A textbook of electrical technology* (Vol. 1: Electrical engineering principles, 25th ed.). S. Chand & Company.
- USB Implementers Forum. (2024). USB 3.2 and USB Power Delivery 3.1 specifications. USB-IF. Retrieved June 12, 2025, from <https://www.usb.org/documents>
- Vincent, I. O., Okonkwo, E. A., & Bello, H. (2023). Reproducibility in undergraduate engineering research: Principles and practices. *Journal of Engineering Education Transformations*, 36(3), 1–16.

## APPENDIX A: CIRCUIT DIAGRAM

The complete schematic diagram of the Smart USB Power Socket is reproduced below. The schematic is identical to that presented as Figure 2.1, and is repeated here for ease of reference in the appendix.

### 4. USB Power Electric Socket



The circuit system of a versatile USB power socket safely converts the 12V battery voltage into a stable 5V.

This circuit system is available at [USB Power Socket](#)

*Appendix A.1: Schematic diagram of the USB Power Electric Socket circuit.*



## APPENDIX B: ARDUINO SOURCE CODE (KEY EXCERPTS)

The following listing presents the key excerpts of the ESP8266 firmware. The complete sketch is approximately 320 lines and is available from the project repository. The excerpts below cover the Wi-Fi connection, HTTP server setup, sensor reading, and relay control functions.

```
// SmartSocket.ino - ESP8266 firmware for Smart USB Power Socket
// Nile University of Technology, Abuja - 2025/2026 Session

#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <ArduinoJson.h>

// ---- Pin assignment ----
const int V_PIN      = A0;      // voltage sense (via 10:1 divider)
const int I_PIN      = A0;      // ACS712 (alternate sample)
const int RELAY_PIN  = D2;      // active-low relay module
const int LED_PIN    = D5;      // status LED

// ---- Calibration constants ----
const float V_DIV_RATIO = 10.0;
const float ACS_SENS    = 0.066;
const float ACS_ZERO    = 2.5;

// ---- Wi-Fi credentials ----
const char* SSID = "SmartHome_2.4G";
const char* PASS = "<redacted>";

ESP8266WebServer server(80);

// ---- History buffer (60 samples @ 1 min) ----
struct Sample { float v, i, p; unsigned long t; };
Sample history[60];
int histIdx = 0;

void readSensors(float &v, float &i) {
    int raw = analogRead(V_PIN);
    v = (raw / 1023.0) * V_DIV_RATIO;
    // ... ACS712 read omitted for brevity
    i = 0.0; // placeholder
}

void handleStatus() {
    float v, i; readSensors(v, i);
    float p = v * i;
    String j = "{\"voltage\":\""+String(v,3)+
```

```
        ", \"current\": "+String(i,3)+
        ", \"power\": "+String(p,3)+
        ", \"relay\": \"\"+(digitalRead(RELAY_PIN)?\"on\":\"off\")+\"\"}";
    server.sendHeader("Access-Control-Allow-Origin", "*");
    server.send(200, "application/json", j);
}

void handleRelay() {
    if (!server.hasArg("plain")) { server.send(400); return; }
    StaticJsonDocument<64> doc;
    deserializeJson(doc, server.arg("plain"));
    String state = doc["state"];
    digitalWrite(RELAY_PIN, state == "on" ? LOW : HIGH);
    server.send(200, "application/json", "{\"ok\":true}");
}

void setup() {
    Serial.begin(115200);
    pinMode(RELAY_PIN, OUTPUT); digitalWrite(RELAY_PIN, HIGH);
    pinMode(LED_PIN, OUTPUT);
    WiFi.mode(WIFI_STA); WiFi.begin(SSID, PASS);
    while (WiFi.status() != WL_CONNECTED) { delay(400); }
    server.on("/status", HTTP_GET, handleStatus);
    server.on("/relay", HTTP_POST, handleRelay);
    server.begin();
}

void loop() {
    server.handleClient();
    static unsigned long lastSample = 0;
    if (millis() - lastSample > 60000) {
        lastSample = millis();
        float v, i; readSensors(v, i);
        history[histIdx] = { v, i, v*i, millis() };
        histIdx = (histIdx + 1) % 60;
    }
}
```

## APPENDIX C: ANDROID KOTLIN SOURCE CODE (KEY EXCERPTS)

The following listing presents the key excerpts of the Android application developed using Google Antigravity. The complete source code is approximately 1,800 lines of Kotlin distributed across 14 files and is available from the project repository. The excerpts below cover the data classes, Retrofit interface, ViewModel, and Jetpack Compose dashboard screen.

```
// SmartSocketApi.kt
package com.nileuniv.smartsocket.data

import retrofit2.http.*

data class SocketStatus(
    val voltage: Float,
    val current: Float,
    val power: Float,
    val relay: String
)

data class RelayCommand(val state: String)
data class PowerSample(val t: Long, val p: Float)

interface SmartSocketApi {
    @GET("status")
    suspend fun getStatus(): SocketStatus

    @POST("relay")
    suspend fun setRelay(@Body cmd: RelayCommand): SocketStatus

    @GET("history")
    suspend fun getHistory(@Query("hours") hours: Int = 24):
    List<PowerSample>
}

// DashboardViewModel.kt
package com.nileuniv.smartsocket.ui

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.flow.*
import kotlinx.coroutines.launch

data class DashboardUiState(
    val voltage: Float = 0f,
    val current: Float = 0f,
```

```
        val power: Float = 0f,
        val relayOn: Boolean = false,
        val isLoading: Boolean = false,
        val error: String? = null
    )

class DashboardViewModel(private val repo: SocketRepository) : ViewModel()
{
    private val _ui = MutableStateFlow(DashboardUiState())
    val ui: StateFlow<DashboardUiState> = _ui.asStateFlow()

    fun refresh() = viewModelScope.launch {
        _ui.update { it.copy(isLoading = true) }
        try {
            val s = repo.getStatus()
            _ui.update {
                it.copy(voltage = s.voltage, current = s.current,
                    power = s.power, relayOn = s.relay == "on",
                    isLoading = false, error = null)
            }
        } catch (e: Exception) {
            _ui.update { it.copy(isLoading = false, error = e.message) }
        }
    }

    fun toggleRelay() = viewModelScope.launch {
        repo.setRelay(!_ui.value.relayOn); refresh()
    }
}
```

## APPENDIX D: BILL OF MATERIALS

The complete bill of materials is presented in Table D.1 below. All components were sourced from the Nigerian electronics market in Abuja during the 2025/2026 academic session. Prices are quoted in Naira and are exclusive of negotiation discounts, which typically reduce the total cost by 10–15%.

**Table D.1: Complete bill of materials with unit and total costs**

| S/N | Component                               | Qty | Unit Cost (₦) | Total (₦) |
|-----|-----------------------------------------|-----|---------------|-----------|
| 1   | LM317 regulator (TO-220)                | 1   | 850           | 850       |
| 2   | 1N4007 diode                            | 1   | 50            | 50        |
| 3   | Electrolytic capacitor 470 $\mu$ F/25 V | 1   | 200           | 200       |
| 4   | Electrolytic capacitor 10 $\mu$ F/25 V  | 1   | 100           | 100       |
| 5   | Electrolytic capacitor 1 $\mu$ F/10 V   | 1   | 80            | 80        |
| 6   | Ceramic capacitor 0.1 $\mu$ F           | 1   | 50            | 50        |
| 7   | Resistor 270 $\Omega$ (1/4 W)           | 1   | 30            | 30        |
| 8   | Resistor 820 $\Omega$ (1/4 W)           | 1   | 30            | 30        |
| 9   | Resistor 330 $\Omega$ (1/4 W)           | 1   | 30            | 30        |
| 10  | Zener diode 5.6 V / 1 W                 | 1   | 150           | 150       |
| 11  | LED 5 mm red                            | 1   | 50            | 50        |
| 12  | USB Type-A female socket                | 1   | 300           | 300       |

| S/N | Component                                   | Qty | Unit Cost (₦) | Total (₦) |
|-----|---------------------------------------------|-----|---------------|-----------|
| 13  | Cigar-lighter plug (12 V)                   | 1   | 600           | 600       |
| 14  | ESP8266 NodeMCU v3                          | 1   | 2,500         | 2,500     |
| 15  | ACS712 current sensor module                | 1   | 1,200         | 1,200     |
| 16  | 5 V relay module (1 channel)                | 1   | 800           | 800       |
| 17  | Voltage divider resistors 10 k $\Omega$ (2) | 1   | 60            | 60        |
| 18  | Breadboard / perfboard                      | 1   | 500           | 500       |
| 19  | Jumper + hookup wire (assorted)             | 1   | 400           | 400       |
| 20  | Plastic enclosure 100×60×30 mm              | 1   | 700           | 700       |
|     | TOTAL                                       |     |               | 8,230     |

Note: The total in Table D.1 reflects the components unique to the smart socket and excludes the Android smartphone used as the controlling device. The Chapter Three figure of approximately ₦14,800 includes additional consumables (solder, flux, thermal paste, mounting hardware) and a 20% contingency allowance.